Docker

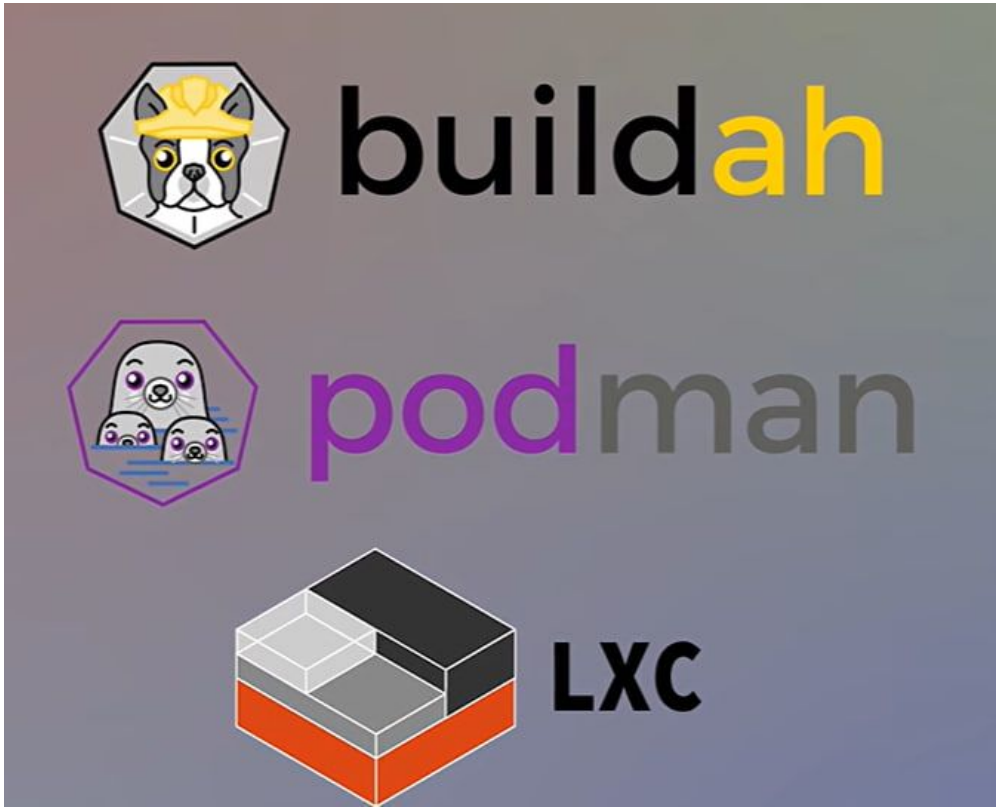
Выполнили студенты АС-23-04:
Баранов И. Д.
Батманов Е.В.
Газдик А.С.



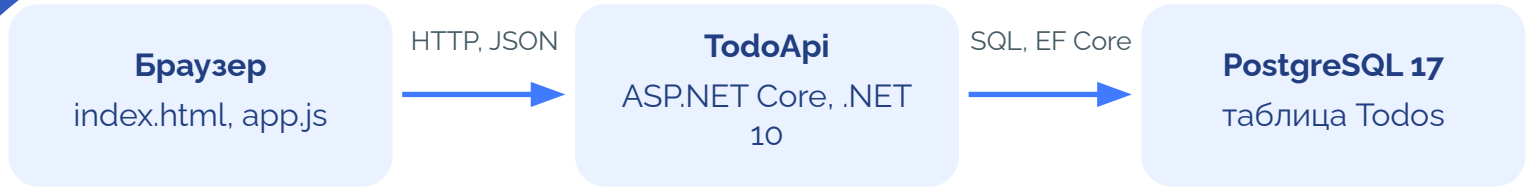
Что такое докер и зачем он нужен

Docker — это платформа, которая упаковывает приложение вместе со всем, что ему нужно для работы (среда выполнения, библиотеки, настройки), в изолированный контейнер. Такой контейнер **запускается одинаково на любой машине** — на любом сервере и тд, — поэтому исчезает проблема «у меня всё работало, а у тебя нет»

Альтернативы Docker



ToDo: браузер, API на .NET 10 и PostgreSQL



Путь	Методы	Что делает
/todoitems	GET, POST	список задач, создание
/todoitems/complete	GET	только выполненные
/todoitems/{id}	GET, PUT, DELETE	одна задача: чтение, изменение, удаление

Коды ответов: 200, 201 Created, 204 No Content, 400 при пустом названии, 404 если задачи нет.

Ключевые преимущества

Без Docker

Установить .NET 10 SDK нужной версии

Установить и настроить PostgreSQL 17

Создать базу, пользователя и пароль

Прописать строку подключения

Повторить на каждом ПК и под каждую ОС

С Docker

Установить только Docker

git clone репозитория

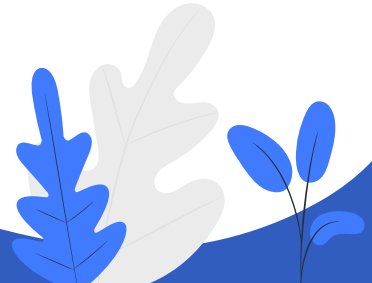
copy .env.example .env

docker compose up

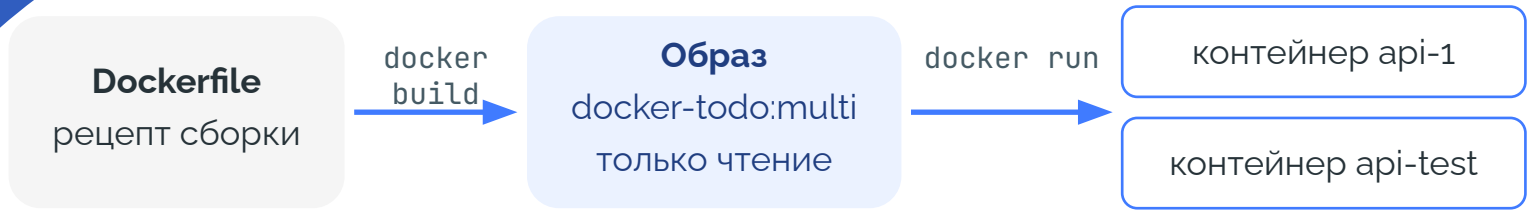
Одинаково на Windows, macOS и Linux

01

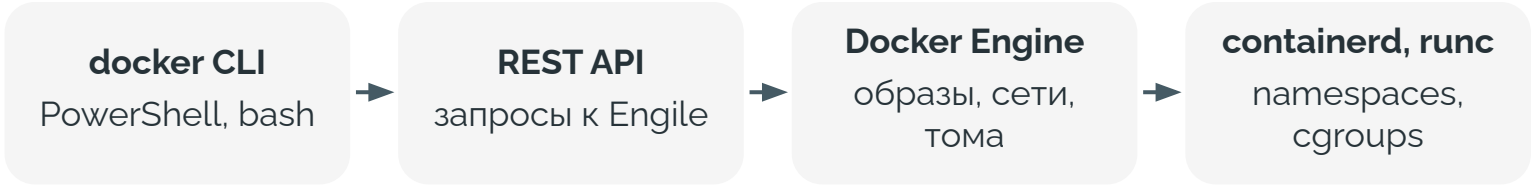
Упаковка приложения



Наша структура



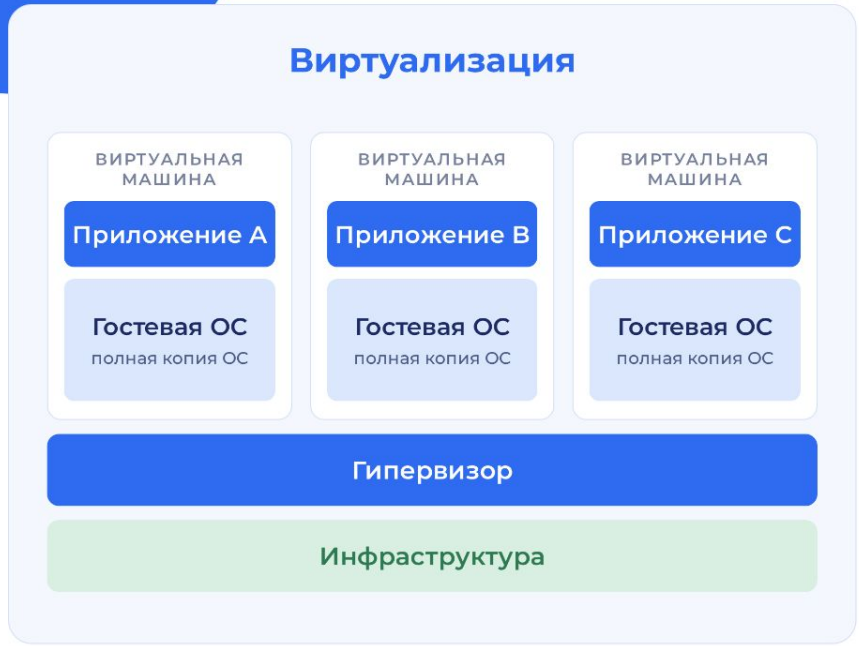
Кто выполняет команды

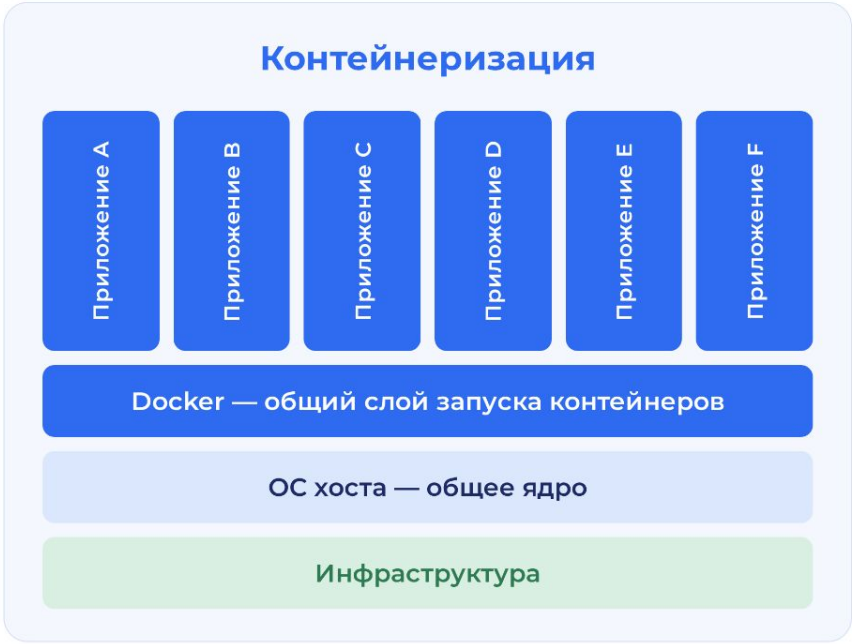


У контейнера свой слой записи поверх образа. Удалили контейнер — слой пропал, образ остался. Поэтому данные PostgreSQL лежат в томе.

Контейнеры и виртуальные машины

Почему контейнер — это легче: сравнение двух подходов к изоляции





Контейнер — это изолированный процесс, а не целая машина: он делит ядро хоста с другими контейнерами. Поэтому наш образ API весит **353 МБ вместе со средой выполнения**, а не гигабайты, и контейнер стартует за секунды.

Program.cs: настройки приходят из окружения

src/TodoApi/Program.cs

```
3  var builder = WebApplication.CreateBuilder(args);
4  var connectionString =
   builder.Configuration.GetConnectionString("TodoDb"
6  );
7  if (string.IsNullOrWhiteSpace(connectionString))
8  {
9      throw new InvalidOperationException(
10         "Не задана строка подключения TodoDb.");
11 }
12 builder.Services.AddDbContext<TodoDb>(options ⇒
13     options.UseNpgsql(connectionString));
14 using (var scope = app.Services.CreateScope())
15 {
       var db =
17 scope.ServiceProvider.GetRequiredService<TodoDb>()
18 ;
19     await db.Database.EnsureCreatedAsync();
20 }
21 app.UseDefaultFiles();
22 app.UseStaticFiles();
```

стр. 3 CreateBuilder читает appsettings.json, затем переменные окружения. Окружение важнее файла.

стр. 4 GetConnectionString("TodoDb") ищет ключ ConnectionStrings:TodoDb. В Docker он приходит из переменной окружения.

стр. 7–11 Нет строки подключения — исключение при старте. Контейнер сразу падает с понятной ошибкой.

стр. 12-16 EnsureCreatedAsync при первом запуске сам создает базу и все таблицы, если их нет. Миграции не нужны.

стр. 21–22 API сам отдаёт index.html и app.js из wwwroot. Отдельный веб-сервер для страницы не нужен.

Что такое **Dockerfile**

Dockerfile- это текстовый файл, содержащий инструкции для создания Docker-образа. В Dockerfile описано:

- Что положить внутрь (библиотеки, код, зависимости);
- Как запускать процесс внутри контейнера (команда при запуске контейнера);
- Какие переменные окружения нужны.

Этап build: компиляция в образе с SDK

docker/egor/Dockerfile

```
1  # Этап 1: сборка, SDK в итоговый образ не
   попадает
2  FROM mcr.microsoft.com/dotnet/sdk:10.0 AS
   build
3  WORKDIR /src
4
5  # Сначала только csproj: пока он не менялся,
   restore берётся из кеша
6  COPY src/TodoApi/TodoApi.csproj .
7  RUN dotnet restore
8
9  # Весь исходный код, включая wwwroot
10 COPY src/TodoApi/ .
11 RUN dotnet publish -c Release -o /app/publish
   --no-restore
```

Каждая инструкция RUN, COPY, WORKDIR создаёт слой. Порядок инструкций определяет, что возьмётся из кеша.

стр. 2 FROM sdk:10.0 AS build — образ с компилятором и NuGet. Имя build нужно, чтобы потом забрать из него файлы.

стр. 3 WORKDIR /src создаёт папку и делает её текущей. Пути внутри образа Linux.

стр. 6–7 Сначала только csproj и restore. Слой с пакетами берётся из кеша, пока не менялись зависимости.

стр. 10–11 Копируем код и публикуем Release в /app/publish. --no-restore: пакеты уже скачаны слоем выше.

Этап final: в образ попадает только runtime

docker/egor/Dockerfile

```
13 # Этап 2: лёгкий runtime без компилятора
14 FROM mcr.microsoft.com/dotnet/aspnet:10.0
   AS final
15 WORKDIR /app
16 COPY --from=build /app/publish .
17
18 # Соглашение команды: API слушает HTTP на
   8080 (Nginx идёт на api:8080)
19 ENV ASPNETCORE_HTTP_PORTS=8080
20 EXPOSE 8080
21
22 # Не от root — пользователь app уже есть в
   образе aspnet
23 USER $APP_UID
24
25 ENTRYPOINT ["dotnet", "TodoApi.dll"]
```

стр. 14 aspnet:10.0 — среда выполнения ASP.NET без SDK и компилятора.

стр. 16 COPY --from=build берёт готовые файлы из первого этапа. Исходники и SDK остаются там.

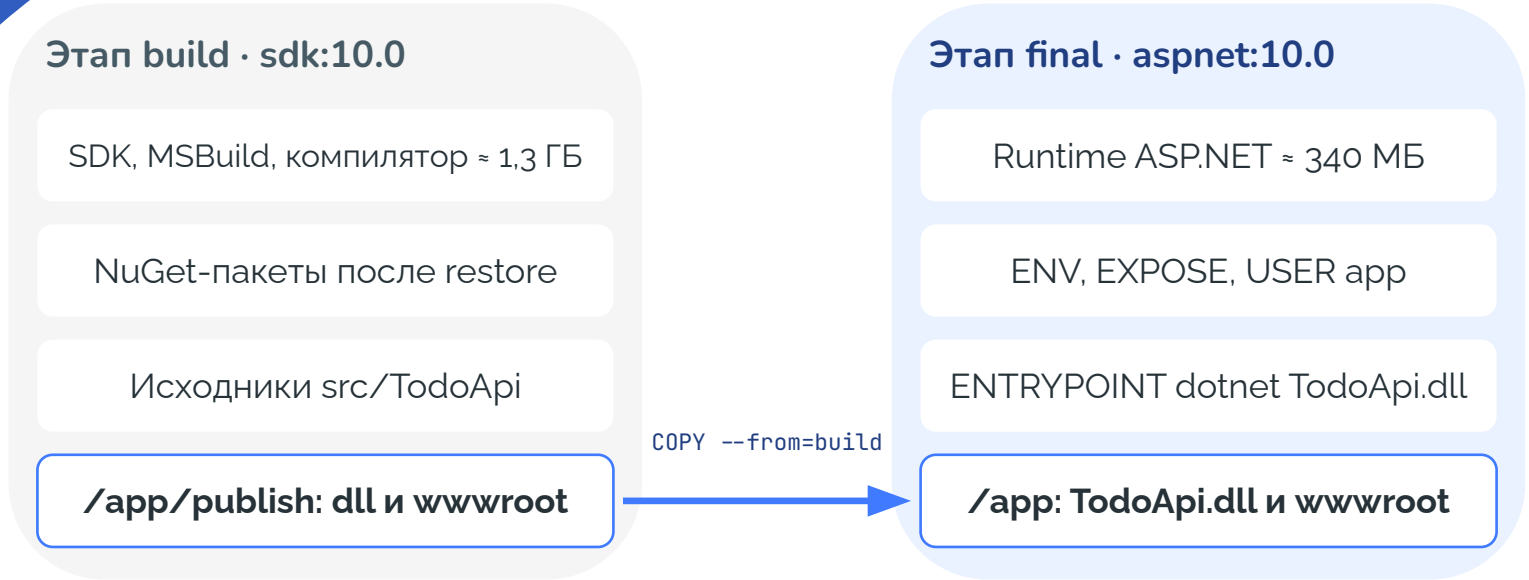
стр. 19 ASPNETCORE_HTTP_PORTS=8080: сервер Kestrel слушает HTTP на 8080.

стр. 20 EXPOSE — только метка. Порт наружу открывают ports в Compose или -p в docker run.

стр. 23 USER $APP_UID: процесс работает не от root. Пользователь app уже есть в образе.

стр. 25 ENTRYPOINT массивом: dotnet запускается без shell и сам получает сигнал остановки.

Многоэтапная сборка: SDK не попадает в образ



Итог: docker-todo:multi ≈ **353 МБ**. Этап build после сборки отбрасывается.

Dockerfile.simple: всё в одном образе с SDK

docker/egor/Dockerfile.simple

```
# Одностадийная сборка — только для сравнения размеров. SDK остаётся в образе.
FROM mcr.microsoft.com/dotnet/sdk:10.0
WORKDIR /app

COPY src/TodoApi/ .
RUN dotnet publish -c Release -o out

# Соглашение команды: API слушает HTTP на 8080
ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "out/TodoApi.dll"]
```

стр. 2 Один этап: итоговый образ — это образ SDK со всем содержимым.

стр. 5 Код копируется целиком сразу. Любая правка сбрасывает кеш, пакеты качаются заново.

стр. 6 publish сам делает restore: пакеты и компиляция в одном слое.

стр. 12 Запуск из out/: рядом остаются исходники, obj и SDK, хотя нужен только runtime.

Файл нужен только для сравнения. В Compose подключён многоэтапный Dockerfile.

Образ multi в 4 раза меньше: 353 МБ и 1,36 ГБ

```
PS C:\ProjectDocker\docker-todo> docker image ls docker-todo

IMAGE                    ID             DISK USAGE   CONTENT SIZE
docker-todo:multi        3d9d92563670       353MB          100MB
docker-todo:simple       18953b7c8e5b      1.36GB          375MB
```

SDK ≈ 1,3 ГБ

нужен только для компиляции и остаётся в этапе build

Быстрее pull

на другой машине скачивается в 4 раза меньше данных

Меньше уязвимостей

в образе нет компилятора и лишних утилит

Контекст сборки: 14,86 КБ вместо всей папки

.dockerignore

```
# Git и файлы IDE — в контексте сборки не нужны
.git
.gitattributes
.gitignore
.vs/
.vscode/
.idea/

# Локальные секреты: пароли не должны попадать в образ
.env
.env.*
!.env.example

# Артефакты локальных сборок .NET
**/bin/
**/obj/

# Инфраструктура и документация — в образе не нужны
docker/
*.md
*.http
```

Перед сборкой CLI отправляет Engine **контекст сборки**— папку, указанную точкой в docker build.

.git, bin/, obj/ тяжёлые и меняются при каждой сборке. Без них COPY не сбрасывает кеш зря.

.env с паролями не попадает в образ. Строка !.env.example возвращает только шаблон.

appsettings.Development.json — настройки разработки остаются на ПК разработчика.

docker/, *.md, *.http приложению при работе не нужны.

Кеш: правка Program.cs не повторяет restore

1 FROM sdk:10.0 AS build	CACHED
2 WORKDIR /src	CACHED
3 COPY TodoApi.csproj .	CACHED
4 RUN dotnet restore	CACHED
5 COPY src/TodoApi/ .	заново
6 RUN dotnet publish	заново

Правило кеша

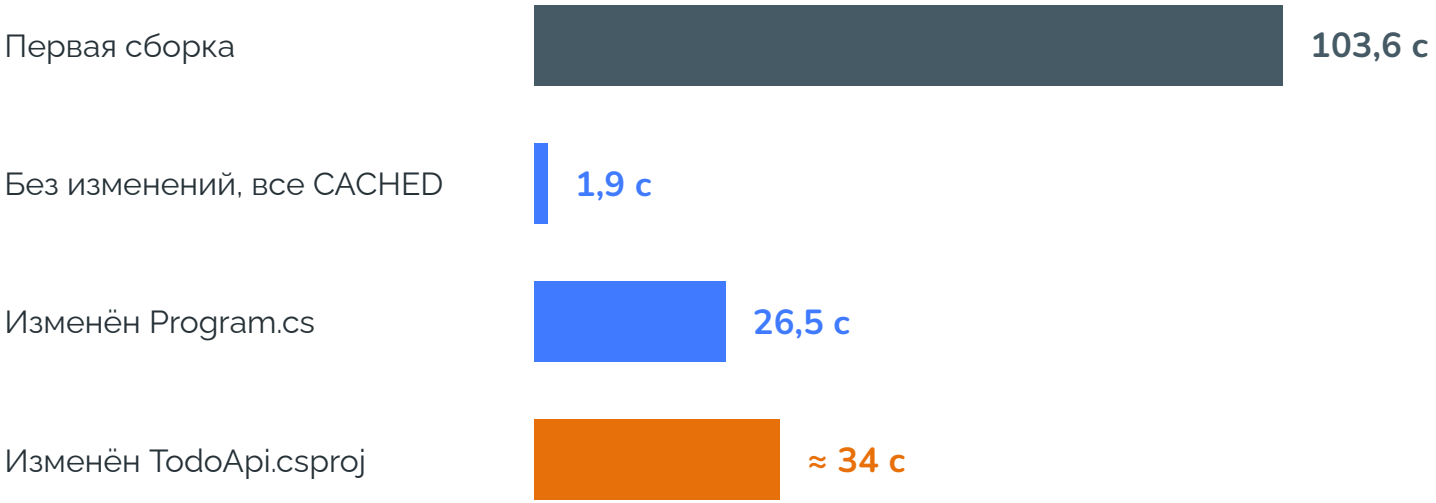
Слой берётся из кеша, если не изменились инструкция и копируемые файлы.

Изменился один слой — пересобираются он и все слои ниже.

Поэтому csproj копируется раньше кода: restore зависит только от него.

Изменили TodoApi.csproj → заново собираются слои 3–6, restore снова ≈ 34 с

Пересборка без изменений: 1,9 с вместо 103,6 с



Program.cs: из 26,5 с на publish ушло 18,8 с, restore взят из кеша. csproj: restore выполняется заново.

Просмотр образов

```
PS C:\ProjectDocker\docker-todo> docker image ls

IMAGE                                  ID             DISK USAGE   CONTENT SIZE   EXTRA
docker-todo:multi                      3d9d92563670       353MB          100MB
docker-todo:simple                     18953b7c8e5b      1.36GB          375MB
mcr.microsoft.com/dotnet/aspnet:10.0   1a48394b6f29       342MB         97.7MB
mcr.microsoft.com/dotnet/sdk:10.0      212fe4cb7462      1.29GB          353MB
postgres:17-alpine                     562a6e0a66b5       424MB          118MB    U
```

DISK USAGE — распакованный размер на диске
CONTENT SIZE — размер сжатых слоёв при скачивании
U — образ используется контейнером.

doker ps и doker ps -a

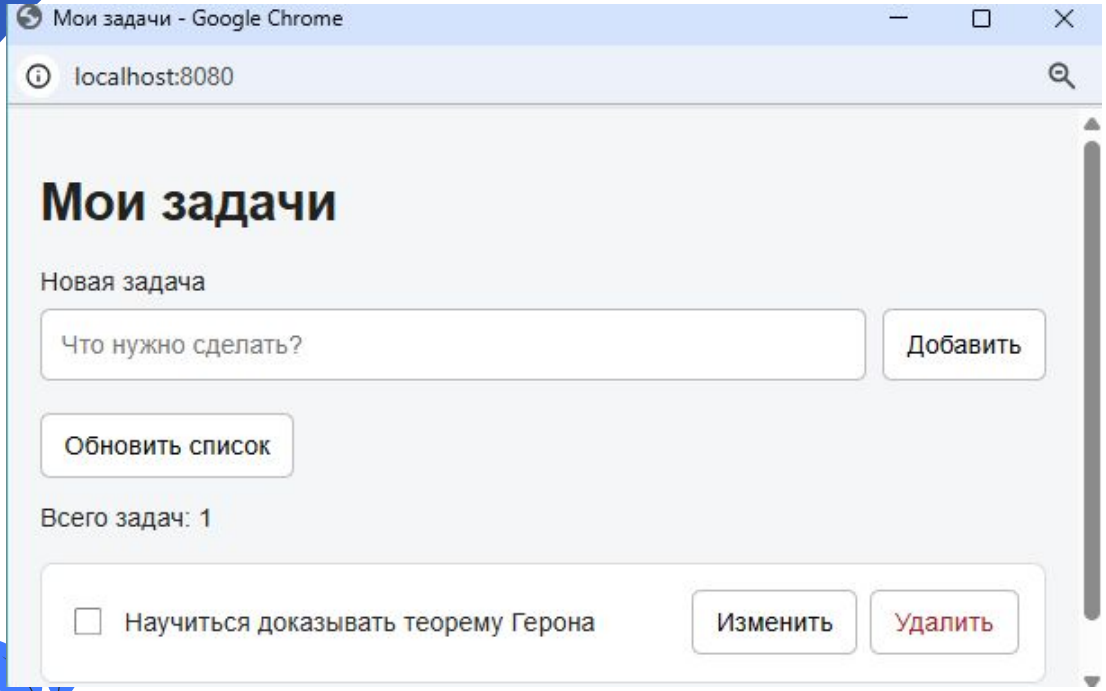
```
PS C:\ProjectDocker\docker-todo> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED      STATUS                    PORTS                    NAMES
476859786b84   postgres:17-alpine   "docker-entrypoint.s…"   5 days ago   Up 10 minutes (healthy)   127.0.0.1:5432->5432/tcp   docker-todo-db-1
PS C:\ProjectDocker\docker-todo> docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED              STATUS                          PORTS                    NAMES
65f9776cbd2a   docker-todo:multi    "dotnet TodoApi.dll"     About a minute ago   Exited (139) About a minute ago                          manual-check
476859786b84   postgres:17-alpine   "docker-entrypoint.s…"   5 days ago           Up 11 minutes (healthy)         127.0.0.1:5432->5432/tcp   docker-todo-db-1
```

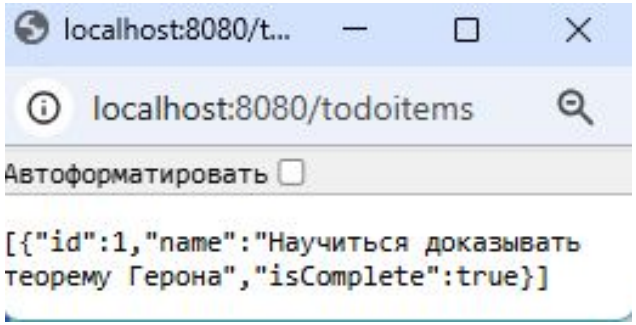
Status: Up 10 minutes — работает; Exited (0) — завершился сам, код 0 = успех; Exited (137) — убит сигналом SIGKILL (так делает docker stop, если процесс не уложился в 10 секунд на graceful shutdown)
Ports: 127.0.0.1:5432->5432/tcp — порт 5432 хоста проброшен в порт 5432 контейнера.
Name: docker-todo-db-1 — имя собирает compose: проект-сервис-номер; при docker run без --name docker придумывает случайное.

```
PS C:\ProjectDocker\docker-todo> docker build -f docker/egor/Dockerfile -t docker-todo:multi .
[+] Building 103.6s (15/15) FINISHED                                                
```

```
PS C:\ProjectDocker\docker-todo> docker build -f docker/egor/Dockerfile -t docker-todo:multi .
[+] Building 1.9s (15/15) FINISHED                                                
```

Проверка: задача из браузера дошла до базы





Почему образ запускается на любой ОС

.NET компилируется в IL

TodoApi.dll не привязан к процессору и ОС. Его выполняет runtime dotnet из образа aspnet.

Внутри образа Linux

Docker Desktop даёт ядро Linux: на Windows через WSL 2, на macOS через виртуальную машину.

Образы для amd64 и arm64

Базовые sdk и aspnet есть под обе архитектуры. Наш образ собран под amd64, для arm64 нужен buildx.

Настройки через окружение

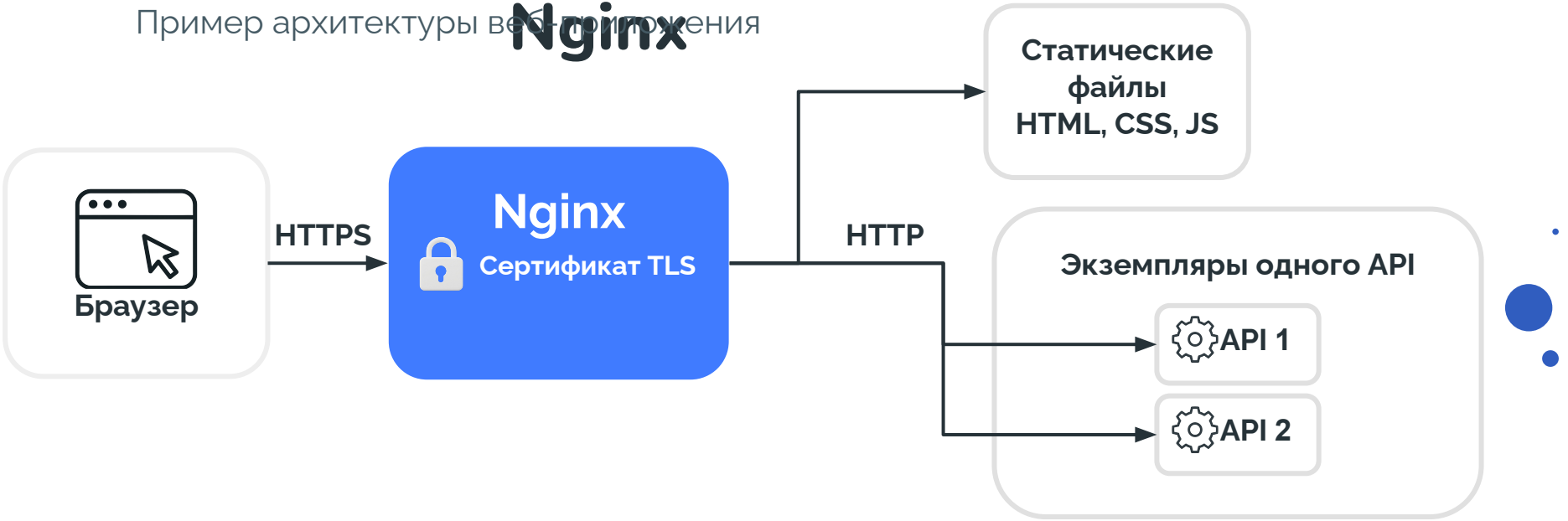
Порт и строка подключения — переменные. В коде нет путей и адресов конкретной машины.

Короткий ответ

Образ несёт всё нужное с собой — **Linux-окружение и runtime .NET**, — а ядро Linux даёт Docker Desktop (**WSL 2** на Windows, виртуальная машина на macOS). Поэтому **ОС компьютера, где запускается контейнер, не важна**.

Для чего используется Nginx

Пример архитектуры веб-приложения



01

HTTPS

Принимает защищенные соединения

02

Статические файлы

Отдает страницы, стили и скрипты, медиа

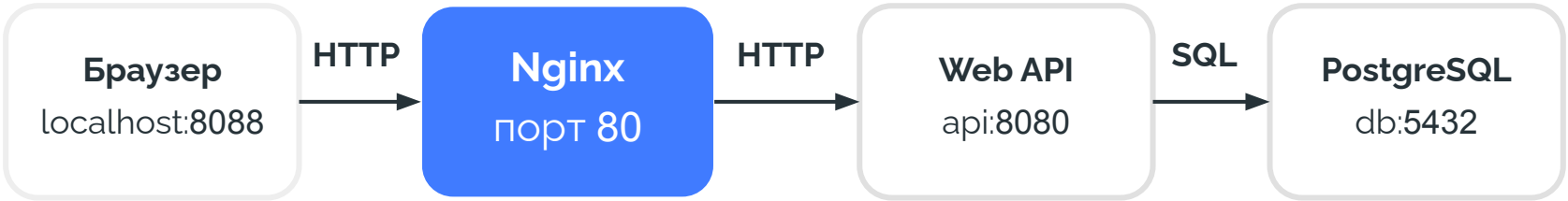
03

Балансировка нагрузки

Распределяет запросы между API

Nginx в нашем проекте

Обратный прокси между браузером и API



01
Принимает запрос

02
Передаёт его API

03
Возвращает ответ браузеру

Настройка Nginx

nginx.conf

```
server {
    listen 80;

    location / {
        proxy_pass http://api:8080;
        proxy_http_version 1.1;

        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

listen 80
Порт внутри контейнера

location / proxy_pass
Все пути направляются на api:8080

proxy_set_header
Хост, IP и схема исходного запроса

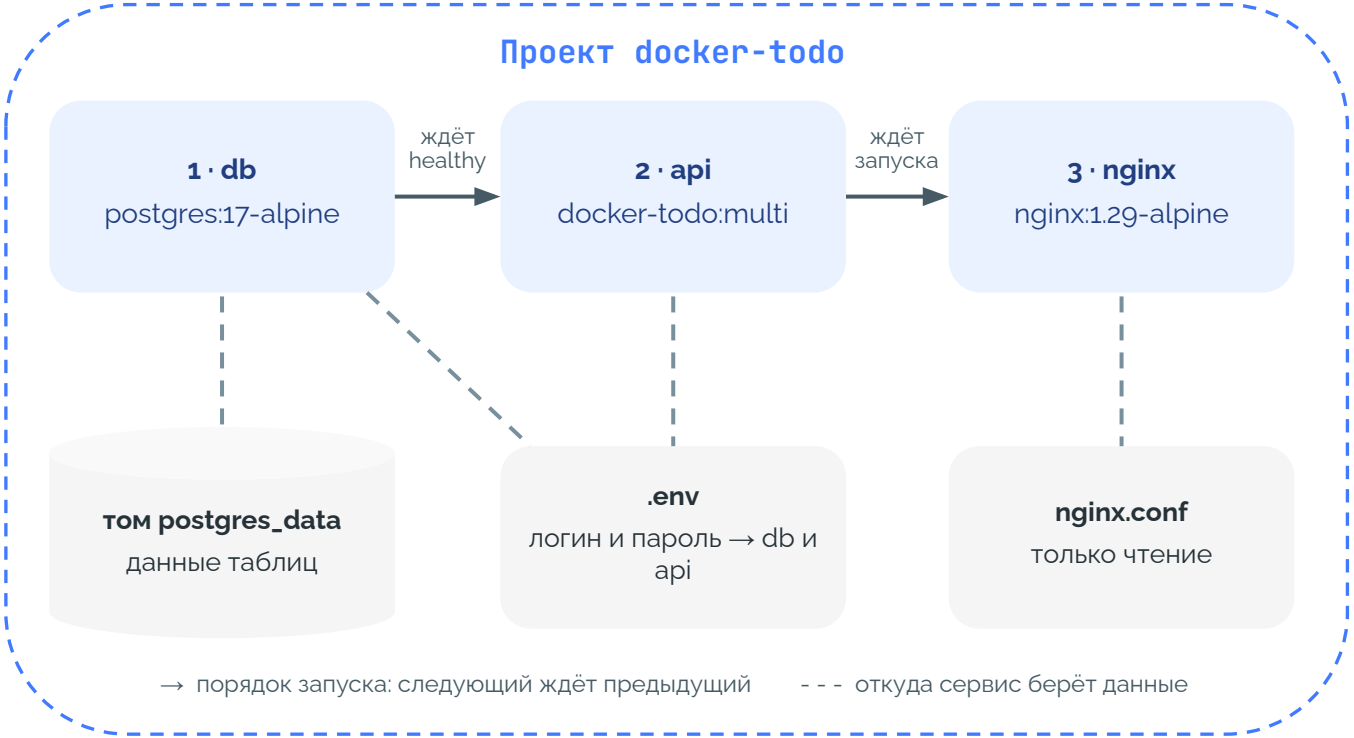
02 Docker compose

Основные команды Docker Compose

Команда	Что делает
`docker compose build`	Собирает образы сервисов с секцией build. У нас — образ api по Dockerfile Егора. Контейнеры не запускает.
`docker compose pull`	Скачивает готовые образы из реестра: postgres:17-alpine и nginx:1.29-alpine. Образ api собирается через build.
`docker compose up -d`	Создаёт сеть, том и контейнеры и запускает их в фоне. Недостающие образы скачивает или собирает.
`docker compose ps`	Показывает контейнеры проекта: состояние, результат healthcheck и порты.
`docker compose logs`	Выводит логи всех сервисов проекта. Перед каждой строкой — имя сервиса.
`docker compose down`	Останавливает и удаляет контейнеры и сеть. Том с данными остаётся.

Все команды запускаются из папки с compose.yaml и действуют на весь проект сразу. Полезные флаги: **`up --build`** — пересобрать образы, **`logs -f`** — следить за логами, **`down -v`** — удалить ещё и том.

Проблема запуска



Настройка вручную

Windows PowerShell · запуск трёх контейнеров вручную

```
PS> docker network create docker-todo_default
PS> docker volume create docker-todo_postgres_data
PS> docker run -d --name db --network docker-todo_default --restart unless-stopped `
      -e POSTGRES_DB=tododb -e POSTGRES_USER=todouser -e POSTGRES_PASSWORD=todo_local_password `
      -v docker-todo_postgres_data:/var/lib/postgresql/data -p 127.0.0.1:5432:5432 `
      --health-cmd "pg_isready -U todouser -d tododb" --health-interval 5s `
      --health-retries 10 --health-start-period 10s postgres:17-alpine
PS> docker inspect -f "{{.State.Health.Status}}" db
PS> docker build -f docker/egor/Dockerfile -t docker-todo:multi .
PS> $cs = "Host=db;Port=5432;Database=tododb;Username=todouser;Password=todo_local_password"
PS> docker run -d --name api --network docker-todo_default --restart unless-stopped `
      -e "ConnectionStrings__TodoDb=$cs" docker-todo:multi
PS> docker run -d --name nginx --network docker-todo_default --restart unless-stopped `
      -p 8088:80 -v "${PWD}/docker/ilya/nginx.conf:/etc/nginx/conf.d/default.conf:ro" `
      nginx:1.29-alpine
```

Без Compose: 8 команд

сеть → том → db → ждать healthy → образ → api → nginx

С Compose: 1 команда

`docker compose up -d --build --wait`

Сервис db

compose.yaml

```yaml
name: docker-todo

services:
  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-tododb}
      POSTGRES_USER: ${POSTGRES_USER:-todouser}
      POSTGRES_PASSWORD:
${POSTGRES_PASSWORD:-todo_local_password}
    ports:
      - "127.0.0.1:5432:5432"
```

стр. 1 name — имя проекта. От него имена сети docker-todo_default и тома.

стр. 5 Официальный образ PostgreSQL 17 на Alpine. Базу мы не собираем.

стр. 6 unless-stopped: поднять после падения и после перезапуска Docker.

стр. 8–11 При первом старте образ создаёт базу и пользователя из переменных. ${X:-default} — значение из .env или запасное.

стр. 12–13 Порт только на 127.0.0.1: для API из Visual Studio. С других компьютеров базы не видно.

Сервис db Healthcheck

compose.yaml

```
14      volumes:
15        - postgres_data:/var/lib/postgresql/data
16      healthcheck:
17        test: ["CMD-SHELL", "pg_isready -U
    $$POSTGRES_USER -d $$POSTGRES_DB"]
18        interval: 5s
19        timeout: 5s
20        retries: 10
21        start_period: 10s
```

Без healthcheck Compose знает только, что контейнер запущен, а не что база готова принимать запросы.

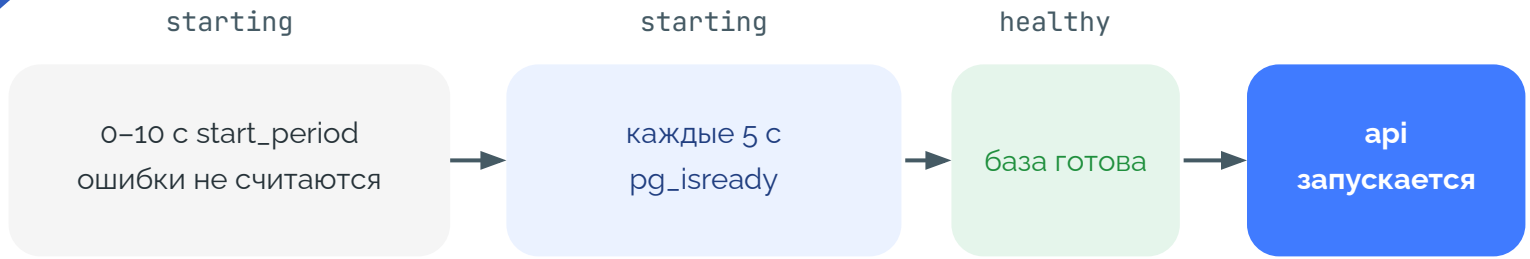
стр. 14–15 Данные PostgreSQL в именованном томе. Контейнер можно удалить, задачи останутся.

стр. 17 pg_isready проверяет, что сервер принимает подключения. CMD-SHELL запускает команду через sh.

стр. 17 $$ — экранирование: Compose не подставляет переменную, её раскрывает sh внутри контейнера.

стр. 18–21 Раз в 5 с, ответ ждём 5 с, 10 неудач подряд — unhealthy. Первые 10 с неудачи не считаются.

порядок старта



10 неудач подряд → **unhealthy** = api не стартует. docker compose up --wait вернёт ошибку.

```
Depends_on: #условие запуска
  db:
    condition: service_healthy
```

Без condition depends_on ждёт только запуска контейнера db. API пытался бы подключиться к базе, которая ещё не готова.

Сервис api

compose.yaml

```yaml
  api:
    build:
      context: .
      dockerfile: docker/egor/Dockerfile
    image: docker-todo:multi
    restart: unless-stopped
    environment:
      ConnectionStrings__TodoDb:
"Host=db;Port=5432;Database=${POSTGRES_DB:-tododb};
Username=${POSTGRES_USER:-todouser};Password=${POST
GRES_PASSWORD:-todo_local_password}"
    depends_on:
      db:
        condition: service_healthy
    # Порты наружу не публикуются: вход только
через nginx.
```

build context . — корень репозитория, dockerfile — файл Егора. **image** — имя готового образа.

ConnectionStrings__TodoDb — двойное подчёркивание .NET читает как двоеточие: ConnectionStrings:TodoDb.

Host=db — имя сервиса, а не localhost. localhost внутри контейнера — это сам api.

depends_on с service_healthy: api стартует после healthy у db.

ports нет — api доступен только внутри сети, через nginx.

Сервис nginx, объявление тома

compose.yaml

```
36   nginx:
37     image: nginx:1.29-alpine
38     restart: unless-stopped
39     ports:
40       - "8088:80"
41     volumes:
42       -
   ./docker/ilya/nginx.conf:/etc/nginx/conf.d/de
43 fault.conf:ro
44     depends_on:
45       - api
46
47 volumes:
     postgres_data:
```

8088:80 — хост:контейнер. Единственный порт, открытый для всех.

./docker/ilya/nginx.conf — файл Ильи вместо стандартного default.conf. **:ro** — только чтение.

depends_on: api — nginx стартует после запуска api, без ожидания healthcheck.

volumes: postgres_data: объявляет именованный том. Docker создаёт его один раз и хранит отдельно от контейнеров.

Что делает docker compose up по шагам

1
Читает compose.yaml и .env, подставляет ${…}

2
--build: собирает образ api по Dockerfile Егора

3
Создаёт сеть docker-todo_default

4
Создаёт том postgres_data, если его нет

5
Запускает db и ждёт статус healthy

6
Запускает api с ConnectionStrings__TodoDb

7
Запускает nginx, публикует порт 8088

8
--wait: возвращает консоль, когда всё готово

Повторный up без изменений ничего не пересоздаёт. Compose сравнивает конфигурацию с работающими контейнерами.

Сеть Compose

встроенный DNS 127.0.0.11

nginx	**api**	**db**
свой IP	свой IP	свой IP

```
запрос из api:
db → IP контейнера db
```

EXPOSE 8080 · Dockerfile

Только метка в образе. С хоста порт не открывается.

ports 8088:80 · compose.yaml

Публикует порт на хосте. Так браузер попадает в nginx.

localhost внутри контейнера — сам контейнер. Поэтому Host=db в строке подключения и api:8080 в nginx.conf.

Запуск стенда

Windows PowerShell · корень репозитория

```
# один раз: шаблон переменных
PS> copy .env.example .env
PS> docker compose --env-file .env up -d --build
--wait
PS> docker compose ps
# ожидаем: db (healthy), api и nginx: Up
PS> docker compose logs --tail 20 api
# ожидаем: Now listening on: http://[::]:8080
# открыть http://localhost:8088
```

Name	Container ID	Image	Port(s)
docker-todo	-	-	-
nginx-1	6472f5c396a6	nginx:1.29-alpine	8088:80
api-1	ee12402bfa59	docker-todo:multi	
db-1	af39cf8dc540	postgres:17-alpine	5432:5432

```
PS> docker compose ... up -d --build --wait
[+] Building 17.5s (17/17) FINISHED
 ⇒ CACHED [build 4/6] RUN dotnet restore
 ⇒ [build 6/6] RUN dotnet publish   12.8s
[+] up 4/4
 ✔ Image docker-todo:multi        Built
 ✔ Container docker-todo-db-1     Healthy
 ✔ Container docker-todo-api-1    Healthy
 ✔ Container docker-todo-nginx-1  Healthy
```

```
PS> docker compose ps
SERVICE  STATUS          PORTS
db       Up (healthy)    127.0.0.1:5432→5432
api      Up              8080/tcp
nginx    Up              0.0.0.0:8088→80
PS> docker compose logs --tail 20 api
api-1  | Now listening on: http://[::]:8080
api-1  | Application started.
```

Разбор вывода docker compose ps

```
PS> docker compose ps
NAME                IMAGE                COMMAND                  SERVICE   CREATED         STATUS                   PORTS
docker-todo-api-1   docker-todo:multi    "dotnet TodoApi.dll"     api       2 minutes ago   Up 2 minutes             8080/tcp
docker-todo-db-1    postgres:17-alpine   "docker-entrypoint.s…"   db        2 days ago      Up 2 minutes (healthy)   127.0.0.1:5432→5432/tcp
docker-todo-nginx-1 nginx:1.29-alpine    "/docker-entrypoint.…"   nginx     2 days ago      Up 2 minutes             0.0.0.0:8088→80/tcp, [::]:8088→80/tcp
```

Столбцы

NAME — имя контейнера: проект-сервис-номер

IMAGE — образ, из которого запущен контейнер

COMMAND — команда запуска внутри контейнера; длинная обрезана «…»

SERVICE — имя сервиса из compose.yaml

CREATED — когда создан контейнер. db и nginx — 2 дня назад: up их не пересоздавал, api пересобран

STATUS — состояние и результат healthcheck

PORTS — внутренние и опубликованные порты

STATUS:

Up 2 minutes — работает 2 минуты с последнего запуска

(healthy) — healthcheck пройден; он есть только у db

(health: starting) — идёт start_period или первые проверки

(unhealthy) — 10 неудач подряд; перезапуска нет

Exited (1) — процесс завершился с ошибкой

PORTS:

8080/tcp — только внутри сети, наружу не опубликован

127.0.0.1:5432→5432/tcp — порт хоста → порт контейнера, только с этого ПК

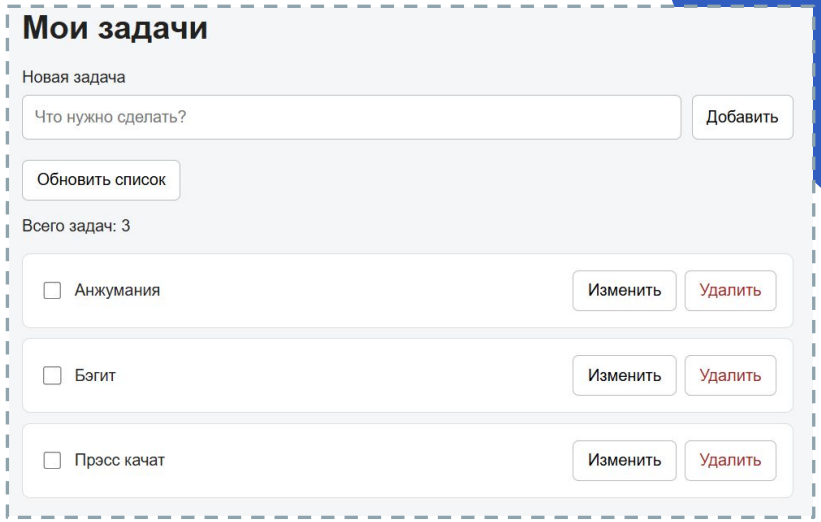
0.0.0.0:8088→80/tcp, [::]:8088→80/tcp — все адреса IPv4 и IPv6, доступно всем

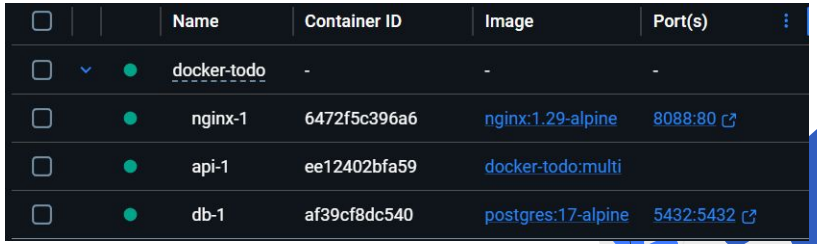
Данные переживают пересоздание контейнеров

Windows PowerShell · проверка тома и сети

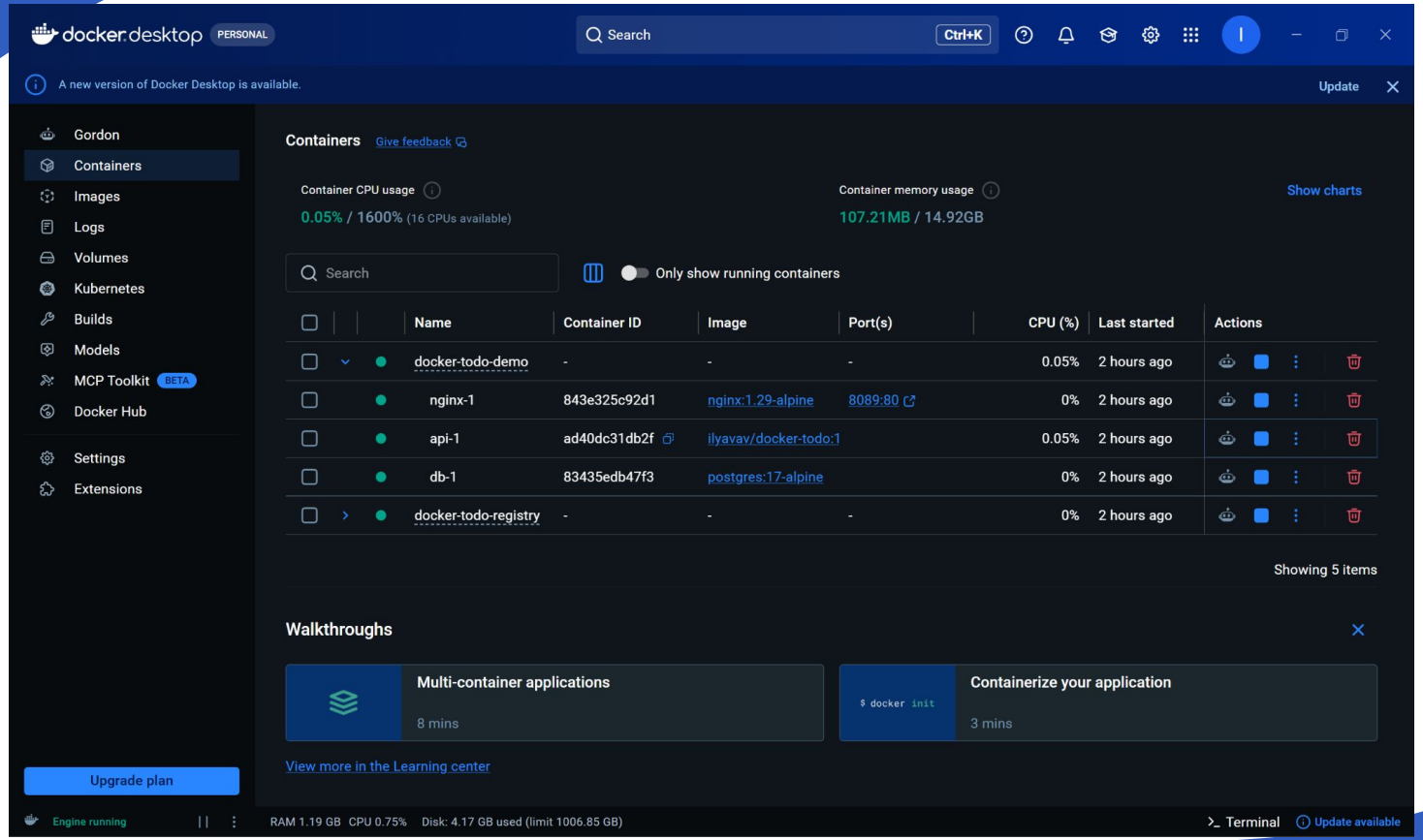
```
# создали задачи на http://localhost:8088
PS> docker compose down
PS> docker volume ls
# docker-todo_postgres_data остался
PS> docker compose --env-file .env up -d --wait
# задачи снова на странице
PS> docker network inspect docker-todo_default
# контейнеры сети и их IP-адреса
```

```
PS> docker network inspect docker-todo_default
"Subnet": "172.18.0.0/16"
"Gateway": "172.18.0.1"
"com.docker.compose.project": "docker-todo"
"Containers": {
  "Name": "docker-todo-nginx-1",
  "IPv4Address": "172.18.0.4/16"
  "Name": "docker-todo-db-1",
  "IPv4Address": "172.18.0.2/16"
  "Name": "docker-todo-api-1",
  "IPv4Address": "172.18.0.3/16"
```





Мониторинг контейнеров в приложении



Отличие down от down -v от stop

Команда	Контейнеры	Сеть	Том	Задачи
docker compose stop	остановлены	остаётся	остаётся	сохранены
docker compose down	удалены	удалена	остаётся	сохранены
docker compose down -v	удалены	удалена	удалён	**потеряны**

Именованный том · db

postgres_data:/var/lib/postgresql/data

Docker сам хранит данные, путь на хосте не важен. Работает одинаково на всех ОС.

Bind mount · nginx

./docker/ilya/nginx.conf:...:ro

Конкретный файл из репозитория. Удобно для конфигов, которые правим вручную.

Когда Docker перезапускает контейнер

Ситуация	Что сделает Docker при unless-stopped
Процесс api завершился с ошибкой	перезапустит контейнер
db получила статус unhealthy	только пометит статус, перезапуска нет
Остановили вручную: docker compose stop	не поднимет, пока не запустим сами
Перезапуск Docker Desktop или ПК	поднимет всё, кроме остановленных вручную

Одна команда запуска на Windows, macOS и Linux

Docker Desktop и WSL 2

Windows — WSL 2, macOS — виртуальная машина, Linux — напрямую. Команды docker одинаковые.

Переводы строк

.gitattributes: * text=auto хранит файлы в Git с LF. Linux-утилиты в контейнерах ждут LF.

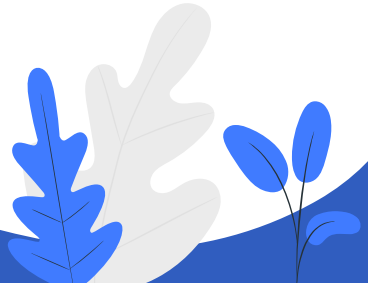
Пути и оболочки

Пути в compose.yaml относительные, через /. Отличаются оболочки: кавычки в PowerShell и bash.

Адреса

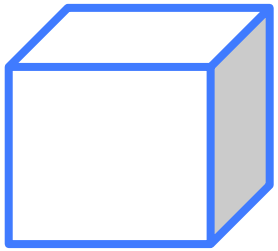
127.0.0.1:5432 закрывает базу от сети на любой ОС. Внутри — имена сервисов, IP хоста не нужен.

03 Docker Registry

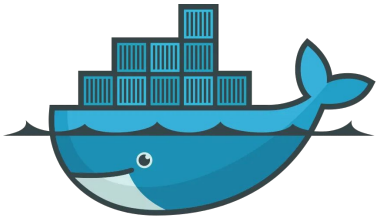


Доставка образа

Передача готового приложения через реестр образов



docker push



docker pull



Docker-образ

Готовый образ приложения

Docker Registry

Хранилище Docker образов

Другой компьютер

Запуск без сборки

Локальный Docker Registry

compose.registry.yaml

```yaml
name: docker-todo-registry

services:
  registry:
    image: registry:3
    restart: unless-stopped
    ports:
      - "127.0.0.1:5000:5000"
    volumes:
      - registry_data:/var/lib/registry

volumes:
  registry_data:
```

image: registry:3
Официальный образ сервера Docker Registry

Ports: 127.0.0.1:5000:5000
Доступ к реестру только с локального компьютера, порт 5000

volumes: registry_data
Постоянное хранение загруженных образов

Запуск и проверка

```
Windows PowerShell

PS C:\Crossplatform\docker-todo> docker compose -f docker/ilya/compose.registry.yaml up -d
[+] up 2/2
 ✔ Network docker-todo-registry_default      Created                                                    0.1s
 ✔ Container docker-todo-registry-registry-1 Started                                                    0.6s
PS C:\Crossplatform\docker-todo> docker compose -f docker/ilya/compose.registry.yaml ps
NAME                              IMAGE      COMMAND                  SERVICE   CREATED       STATUS       PORTS
docker-todo-registry-registry-1 registry:3 "/entrypoint.sh /etc…" registry  5 seconds ago Up 4 seconds 127.0.0.1:5000→5000/tcp
PS C:\Crossplatform\docker-todo> curl.exe -i http://localhost:5000/v2/
HTTP/1.1 200 OK
Content-Length: 2
Content-Type: application/json
Docker-Distribution-Api-Version: registry/2.0
Date: Mon, 09 Nov 2026 17:42:36 GMT

{}
```

docker compose up -d → создание и запуск контейнера Registry

docker compose ps → проверка состояния контейнера и опубликованного порта

curl.exe -i → проверка доступности HTTP API реестра

Публикация образа в

```
Windows PowerShell

PS C:\Crossplatform\docker-todo> docker image ls docker-todo

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker-todo:multi    a5deb6d1e44d        353MB          100MB    U
PS C:\Crossplatform\docker-todo> docker tag docker-todo:multi localhost:5000/docker-todo:demo
PS C:\Crossplatform\docker-todo> docker push localhost:5000/docker-todo:demo
The push refers to repository [localhost:5000/docker-todo]
7a48b8f0e5c7: Pushed
6999a63fda37: Pushed
186601865ce2: Pushed
demo: digest: sha256:a5deb6d1e44d70c7ea43a2566329133e579e4a71793eedd1923ebc6684aa2824 size: 856
```

docker image ls docker-todo → проверка локального образа приложения

docker tag → присвоение образу имени с адресом Registry

docker push → загрузка образа в Registry

Получение и проверка

```
Windows PowerShell

PS C:\Crossplatform\docker-todo> curl.exe http://localhost:5000/v2/docker-todo/tags/list
{"name":"docker-todo","tags":["demo"]}
PS C:\Crossplatform\docker-todo> docker pull localhost:5000/docker-todo:demo
demo: Pulling from docker-todo
Digest: sha256:a5deb6d1e44d70c7ea43a2566329133e579e4a71793eedd1923ebc6684aa2824
Status: Downloaded newer image for localhost:5000/docker-todo:demo
localhost:5000/docker-todo:demo
PS C:\Crossplatform\docker-todo> docker image ls localhost:5000/docker-todo

IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
localhost:5000/docker-todo:demo    a5deb6d1e44d       353MB          100MB    U
```

curl.exe → проверка опубликованного тега в Registry

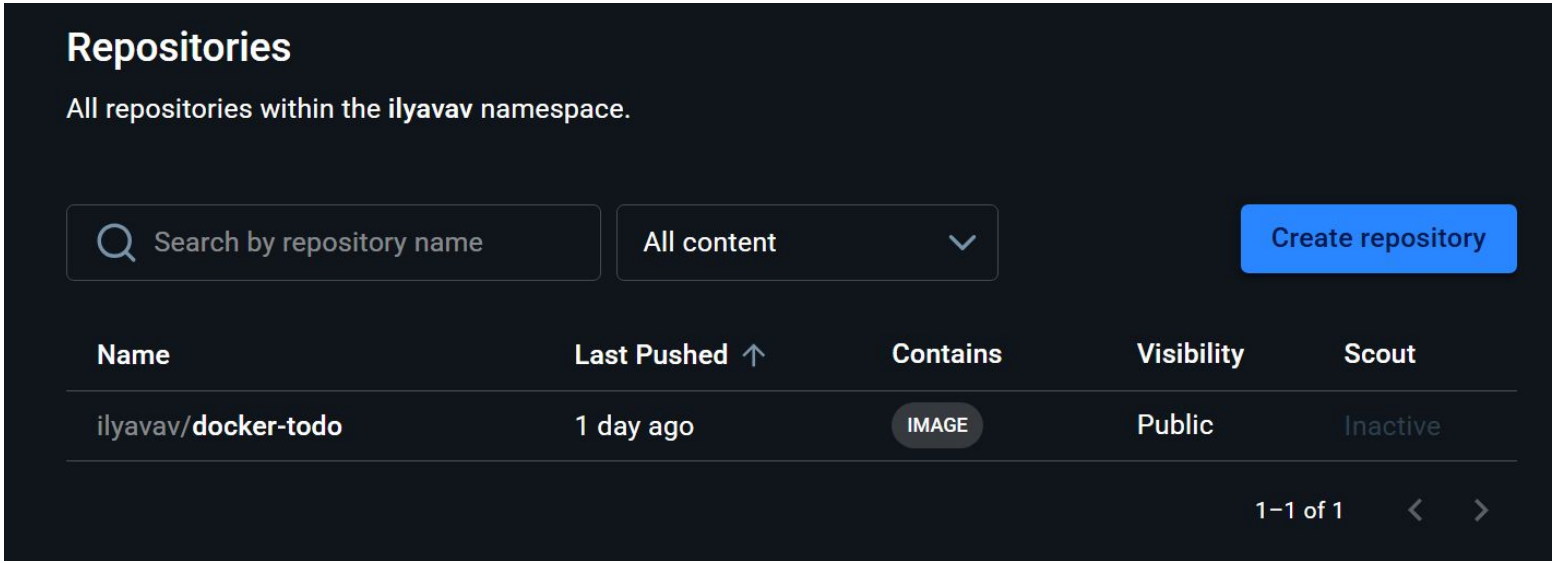
docker pull → получение образа из Registry

docker image ls → проверка образа в локальном хранилище



Docker Hub

Docker Hub - это удаленный сервис для хранения и распространения Docker-образов.



Создан публичный репозиторий **ilyavav/docker-todo** для нашего приложения

Публикация образа в Docker

```
Windows PowerShell

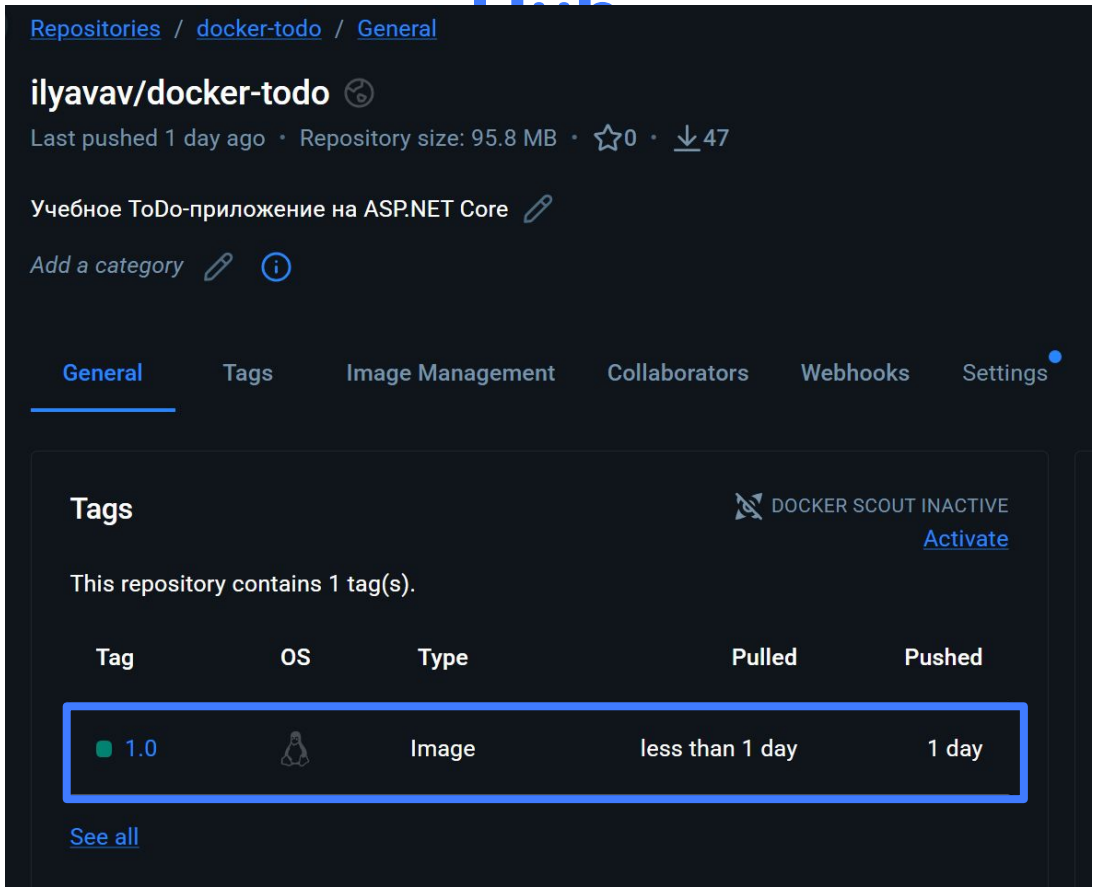
PS C:\Crossplatform\docker-todo> docker login
Authenticating with existing credentials... [Username: ilyavav]
Login Succeeded
PS C:\Crossplatform\docker-todo> docker tag docker-todo:multi ilyavav/docker-todo:1.0
PS C:\Crossplatform\docker-todo> docker push ilyavav/docker-todo:1.0
The push refers to repository [docker.io/ilyavav/docker-todo]
36e06772f126: Layer already exists
b7210ef9e432: Layer already exists
186601865ce2: Layer already exists
1.0: digest: sha256:a5deb6d1e44d70c7ea43a2566329133e579e4a71793eedd1923ebc6684aa2824 size: 856
```

docker login → авторизация в Docker Hub

docker tag → присвоение имени ilyavav/docker-todo:1.0

docker push→ публикация образа в Docker Hub

Образ в Docker Hub



Образ успешно опубликован

ilyavav/docker-todo:1.0

Доступен для загрузки из Docker Hub

Развертывание из готовых образов

compose.images.yaml (фрагмент)

```yaml
name: docker-todo-demo

services:
  db:
    image: postgres:17-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data

  api:
    image: ilyavav/docker-todo:1.0

  nginx:
    image: nginx:1.29-alpine
    ports:
      - "8089:80"

volumes:
  postgres_data:
```

image вместо build

Запуск API из готового образа Docker Hub без повторной сборки

docker-todo-demo

Отдельный Compose-проект со своими контейнерами и томом БД

8089:80

Доступ к приложению через Nginx на порту 8089

Запуск отдельного стенда

```
Windows PowerShell

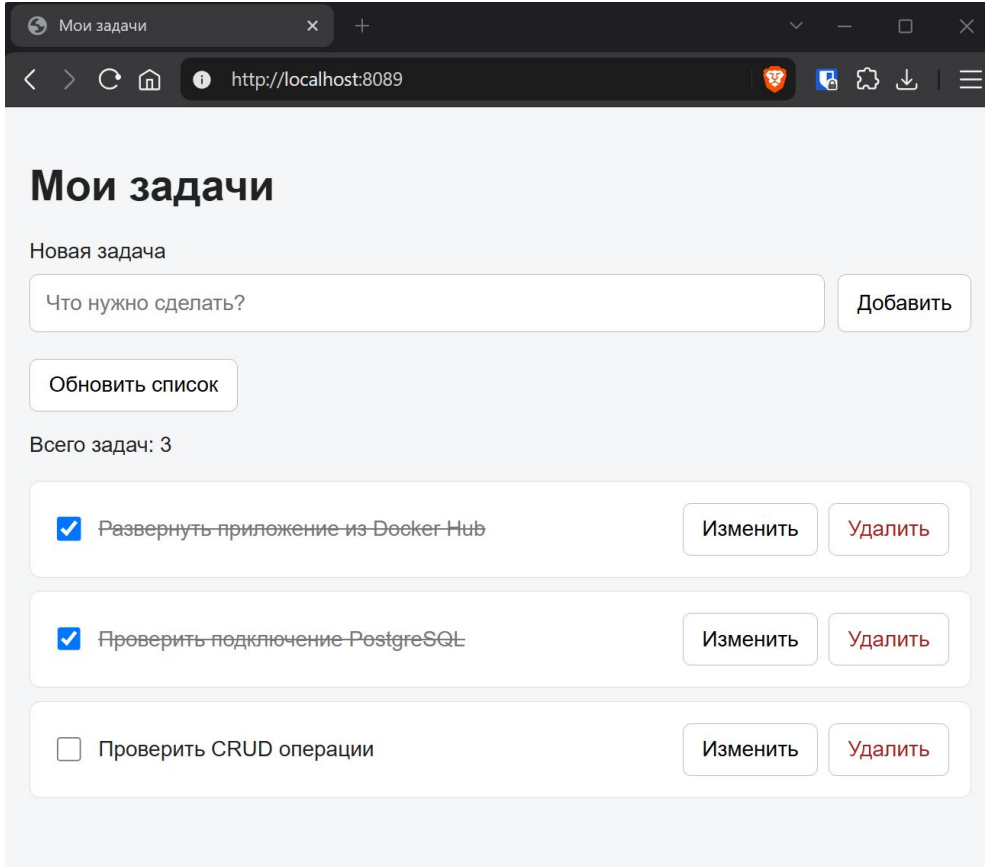
PS C:\Crossplatform\docker-todo> docker compose --env-file .env -f docker/ilya/compose.images.yaml pull
[+] pull 3/3
 ✔ Image nginx:1.29-alpine       Pulled                                                                    1.5s
 ✔ Image postgres:17-alpine      Pulled                                                                    1.7s
 ✔ Image ilyavav/docker-todo:1.0 Pulled                                                                    1.5s
PS C:\Crossplatform\docker-todo> docker compose --env-file .env -f docker/ilya/compose.images.yaml up -d
[+] up 3/3
 ✔ Container docker-todo-demo-db-1    Healthy                                                              5.8s
 ✔ Container docker-todo-demo-api-1   Started                                                              0.3s
 ✔ Container docker-todo-demo-nginx-1 Started                                                              0.4s
PS C:\Crossplatform\docker-todo> docker compose --env-file .env -f docker/ilya/compose.images.yaml ps
NAME                     IMAGE                     COMMAND                  SERVICE CREATED       STATUS         PORTS
docker-todo-demo-api-1   ilyavav/docker-todo:1.0 "dotnet TodoApi.dll"   api     34 hours ago  Up 5 seconds   8080/tcp
docker-todo-demo-db-1    postgres:17-alpine      "docker-entrypoint.s…" db      34 hours ago  Up 11 seconds  5432/tcp
docker-todo-demo-nginx-1 nginx:1.29-alpine       "/docker-entrypoint.…" nginx   34 hours ago  Up 5 seconds   0.0.0.0:8089→80/tcp
```

docker compose pull → получение готовых образов

docker compose up -d → запуск трёх сервисов

docker compose ps → проверка состояния контейнеров

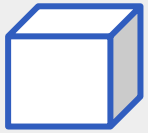
Результат развертывания



Отдельный стенд запущен на localhost: 8089

Для запуска API использован готовый образ из Docker Hub. Повторная сборка приложения не потребовалась.

Итоги проекта



Приложение упаковано в Docker-образ



API, PostgreSQL и Nginx объединены через Docker Compose



Образ опубликован в локальном Registry и Docker Hub. Приложение повторно развернуто без сборки API

https://github.com/ilyavav/docker-todo